

算法设计与分析课程作业指南

小作业要求

一 作业目标与选题

每人从知识点表中认领 1 项，独立完成一份面向同学的微课作业，做到“讲清原理、分析例题、实现算法、验证结果”。选题统一登记，原则上不重复。先讲知识点，再用例题说明，避免只讲一道题的解法。

二 必交材料与要求

1. **讲课 PPT**：提交可编辑 .pptx，建议 8—12 页。包含选题与学习目标、问题定义、核心思想、关键步骤、例题推演、正确性说明、时间与空间复杂度、总结及参考来源。
2. **讲解视频**：提交约 10 分钟的 .mp4。由本人讲解，声音清楚、画面和文字可辨；结合 PPT 展示例题推演及程序运行结果，避免逐页照读。
3. **例题与解析**：至少提供 3 道完整例题，写明题目来源、输入输出格式、数据范围及样例；解析须包含建模思路、关键步骤或状态变化、正确性依据、复杂度和易错点。可单列文档，也可在 PPT 中明确标注对应页。
4. **注释代码与测试**：提交可直接运行的源代码，语言自选。注释解释变量或状态含义、关键步骤的原因、边界处理；附运行环境、执行方法及至少 3 组含预期结果的测试，覆盖常规与边界情况。理论类选题可实现验证或实验程序，并说明其不能替代理论证明。

三 自主设计与学术规范

可增加算法可视化、不同方法对比、应用案例或拓展练习，形式自定，不作为必交项。材料须由本人理解并独立组织，引用题目、图片、代码或使用 AI 辅助均应注明来源及用途；不得直接提交他人成品，须能解释全部关键代码。

四 提交方式

打包命名为“学号_姓名_知识点编号.zip”，内含 PPT、视频、例题解析、源代码与测试、README（运行说明及材料索引）。截止时间和提交平台以教师通知为准。

编号	所属大章节	知识点	大概知识内容
01	算法分析基础	渐近符号与复杂度分析	理解 O 、 Ω 、 Θ ，区分最好、最坏和平均情形；分析基本操作次数及时间、空间开销。例题可选循环或排序程序。
02	图搜索	广度优先搜索 BFS	掌握队列、分层遍历与访问标记，理解首次到达的距离性质；分析邻接表实现的复杂度。例题可选无权图最短路。
03	图搜索	深度优先搜索 DFS	掌握递归或显式栈、访问状态与回溯过程，理解搜索树和连通性判定；例题可选连通分量或有向环检测。
04	图搜索	拓扑排序	理解有向无环图、偏序与拓扑序；掌握入度法及判环条件，说明拓扑序不一定唯一。例题可选课程先修关系。
05	贪心算法	贪心策略设计与适用条件	理解逐步选择、局部最优与全局最优，识别贪心选择性质和最优子结构；比较候选策略并构造反例。以区间调度为例实现一种策略。
06	贪心算法	贪心算法的正确性证明	掌握交换论证和最优值下界两类证明思路，选择一种重点讲解；以最小化延迟调度或区间划分为例，把证明对应到实现步骤。
07	分治算法	分治框架与递归设计	明确子问题规模、递归参数、终止条件及返回值，说明如何由子问题解得到原问题解；以归并排序展示递归调用树与执行过程。
08	分治算法	归并算法与分治合并策略	重点讲解如何在线性时间合并两个有序序列，维护指针与循环不变式，处理相等元素及剩余区间；逆序对计数可作为合并时统计信息的例题。
09	分治算法	分治递推式与主定理	根据子问题个数、规模和非递归工作建立递推式；用递归树与主定理求解，检查适用条件。以归并排序等为分析实例，配操作计数实验。
10	动态规划	最优子结构与状态转移设计	理解最优子结构和重叠子问题；定义状态、推导转移、设置边界并说明正确性。加权区间调度、背包、字符串对齐任选一例。
11	动态规划	记忆化搜索与递推实现	比较自顶向下记忆化和自底向上填表，依据依赖关系确定计算顺序；分析状态数与转移代价。以加权区间调度演示两种实现。
12	网络流	残量网络与增广路算法	理解容量约束、流量守恒、残量容量和反向边；掌握 Ford - Fulkerson 思想及 BFS 增广实现，以小型流网络演示更新。
13	网络流	最大流最小割定理	理解割容量、流值上界及最优性条件；在给定最大流的残量图中求源点可达集，提取最小割并验证定理。

编号	所属大章节	知识点	大概知识内容
14	高级数据结构	并查集	掌握集合代表元、查找与合并，理解路径压缩和按秩或大小合并；区分摊还与单次复杂度。例题可选加边连通性查询。
15	高级数据结构	树状数组	理解 lowbit、节点覆盖范围与前缀分解；实现单点增加、前缀和及区间和，分析下标约定和 $O(\log n)$ 操作复杂度。
16	高级数据结构	线段树与懒标记	理解区间分解、建树、查询与信息合并；掌握懒标记的含义、更新和下传。以区间加、区间求和演示实现。
17	字符串匹配	KMP 算法与失配函数	理解最长相等真前后缀，构建 next 或前缀函数；掌握失配回退、重叠匹配与下标约定，解释线性时间。
18	字符串匹配	Trie 前缀树	理解共享前缀的树形存储；实现插入、完整单词查询与前缀查询，区分路径存在和单词结束，分析操作及存储开销。
19	图优化算法	Kruskal 最小生成树算法	理解割性质与安全边；按边权排序，用并查集判环并选边，说明正确性、复杂度以及非连通图的处理。
20	图优化算法	Prim 最小生成树算法	理解从顶点集合逐步扩展生成树的过程；维护跨割候选边，掌握优先队列实现，分析复杂度及与 Kruskal 的区别。
21	图优化算法	Dijkstra 最短路径算法	理解非负边权条件、松弛与贪心定点；实现优先队列版本及路径还原，处理过期队列项和不可达顶点。
22	图优化算法	Bellman - Ford 算法	掌握多轮边松弛及其含义，处理负权边并检测源点可达负环；说明适用范围、终止条件和 $O(VE)$ 复杂度。
23	计算复杂性	P NP 与多项式归约	区分求解与验证、NP 难与 NP 完全；掌握归约方向及证明要求。独立集与顶点覆盖的互补关系用作归约例题。
24	高级算法专题	近似算法与近似比分析	理解可行解、最优值和近似保证，利用最优值下界推导性能界；以相同机器上的列表调度为例，区分理论界与实测比值。
25	高级算法专题	随机化算法与成功概率分析	理解随机选择、成功概率及重复运行的作用；以随机收缩最小割演示算法，区分实验成功率与理论概率保证。
26	高级算法专题	在线算法与竞争分析	理解信息逐步到达、即时决策和离线最优基准；分析竞争比。以滑雪租赁为例设计阈值策略并分类证明性能界。

课题一

算法设计与分析课程评审平台的设计与实现

一、任务内容

设计并实现面向《算法设计与分析》课程的综合评审平台，供教师和学生使用，支持算法编程题自动测评、主观题匿名互评、学习过程分析和异常评审检测。系统须体现算法设计、复杂度分析和实验验证，不能仅实现常规信息管理功能。

二、功能要求

1. 教师端。至少支持以下功能：

(1) **题库与作业管理**：创建和管理编程题、算法分析题、证明题及开放性问答题；配置测试数据、时间与空间限制、评分规则、评审人数和截止时间。

(2) **学习过程与课程分析**：查看答题情况、提交记录、通过率、提交次数、程序运行时间和主观题评分；统计班级题目通过率、成绩分布、常见错误、题目难度及知识点掌握情况。

(3) **评审过程管理**：查看 Peer Review 分配与评分结果，标记、复核异常或疑似不公平评分，并进行必要的人工修正。

2. 学生端。至少支持以下功能：

(1) **自动评测与学习记录**：支持代码提交、自动编译和测试，反馈正确性、运行时间、空间使用，以及 Accepted、Wrong Answer、Time Limit Exceeded、Runtime Error 等结果；查看个人历史提交、错误记录和题目完成情况。

(2) **主观题提交与匿名互评**：在线提交算法设计说明、复杂度分析、证明等内容，匿名分配给其他学生；评审者按统一标准给出分数与文字意见，被评审者在评审结束后查看汇总结果和修改建议。

三、Peer Review 与算法设计要求

匿名互评不能只采用简单随机分配，应兼顾任务公平性、评分可信度和异常行为检测。

1. **评审分配**：每份作业获得数量基本一致的评价，每名学生承担相近工作量，避免自评和固定互评关系；可考虑跨班级或小组分配，利用随机、匹配或图算法提高公平性。

2. **评分聚合**：设计平均分、去除极端值后的平均、加权平均或基于评审者可信度的动态加权等方案，通过实验比较稳定性与公平性。

3. **异常行为检测**：识别长期偏高或偏低评分、单次评分偏离、异常评审时长及稳定的异常互评关系；可通过降低权重、重新分配或教师复核处理。

项目应至少选择 2~3 个核心问题进行较完整的算法设计，如评审任务分配、评分聚合、异常评分检测、学生能力估计、题目难度分析、相似代码检测或学习行为分析。给出算法思想、伪代码或流程描述，完成时间复杂度、空间复杂度及实验效果分析。

四、实验与成果要求

1. **实验验证**：使用模拟或实际课程数据测试系统与核心算法；鼓励比较不同方法在运行效率、评分稳定性、公平性和异常检测效果上的差异。

2. **提交成果**：可运行的软件系统、完整源代码、系统设计说明、核心算法设计与复杂度分析、测试数据及实验结果。评价重点是问题建模、算法设计、复杂度分析和实验验证。

3. **可选拓展**：图模型异常互评关系检测、评审可信度建模、算法复杂度运行曲线可视化、自动学习报告、代码相似性检测和个性化练习推荐。

课题二

面向算法考试的受限型编程与算法问答大模型

一、任务内容

基于现有轻量级大语言模型，设计并实现面向《算法设计与分析》课程考试或练习的受限型编程与算法问答模型。定位为辅助型智能助教，帮助学生理解程序错误、分析已有代码、解释算法知识并提供适度提示，不能直接替学生完成完整算法题。

可选择合适的开源轻量模型，通过模型微调、LoRA、Prompt 设计、推理 Harness、输入分类、输出审核等方式改造和限制模型能力。

二、模型功能要求

- 编程辅助：**解释编译及运行时错误，定位明显 Bug 或边界问题，分析已有算法的时间与空间复杂度，判断超时或空间不足风险，并提出局部改进建议。可以指出问题和修改方向，应避免直接补全整道题目的核心代码。
- 算法问答与思路引导：**解释基本算法、数据结构及复杂度概念；结合学生已有思考，提示输入规模、复杂度上限、单调性、重复子问题或适用数据结构。对新题不得输出完整题解、完整证明或可直接提交的程序。
- 分级辅助：**可自行设置知识解释、问题诊断、方向提示、局部提示和完整解答等等级；考试场景按规则限制最高帮助等级。

三、模型能力限制与安全要求

限制不能只依赖一句“禁止给出答案”的系统 Prompt，应形成较完整的输入判断、模型生成和输出审核机制。

- 请求识别与回答控制：**区分正常知识问答、程序调试、算法提示、直接索取答案和高风险越狱请求，控制可提供的信息量；可采用意图分类、风险预测、规则与模型联合判断、受限生成或输出二次审核。
- 越狱与答案泄漏防护：**针对角色扮演、忽略系统规则、拆分完整题目、连续追问核心步骤、伪装代码补全或翻译等绕过方式，构建专门安全测试数据并设计防御；重点检测多轮对话中逐步泄漏完整答案的问题，不能仅判断单次输入。

四、Harness 与实验要求

- 自动化评测：**实现批量测试 Harness，数据至少覆盖正常知识问答、代码调试、合法算法提示、直接索取答案及不同形式的越狱攻击。
- 评价维度：**同时评价辅助能力（正确解释、定位错误、有效提示）和安全能力（阻止完整题解、关键算法步骤或可提交代码泄漏）。
- 指标与对比：**可使用正常请求回答正确率、合法辅助完成率、禁止请求拦截率、答案泄漏率、越狱成功率及误拒绝率；比较仅用 Prompt、Prompt 加规则、增加输入分类器、加入输出审核、进一步微调等方案。项目不能仅实现“大模型 API + 聊天界面”。应至少在请求分类、风险识别、帮助等级控制、Prompt Injection 检测、答案泄漏检测、模型微调、受限生成或自动化安全评测等方向中选择若干问题，进行实质性设计和实验。

五、成果要求

- 提交成果：**可运行的模型或原型系统、完整源代码、模型配置或训练代码、Prompt 与 Harness 配置、测试数据集、安全测试集及实验分析报告。
- 评价重点：**基础模型规模不是重点；围绕“帮助学生完成思考，而不是替学生完成算法题”，考察问题建模、模型或算法设计、安全测试和实验分析。
- 可选拓展：**动态辅助等级、基于已有代码的局部帮助、多轮风险累计、自动生成引导性问题、教师端可配置考试策略、自动化 Red Team 越狱测试。